SafeYatra — Product Requirements Document

Smart Tourist Safety Monitoring System — InnoHack DIOT-04 (SDG 10 & 11)

---

0. How to use this document

This is a build-from-zero PRD for both bands and the shared platform. It's organized so each section is buildable in parallel by a different person on your team. Sections 1–2 are scope/requirements, 3–6 are the actual build (hardware → firmware → backend → ML), 7 is integration, 8 is the demo script, 9 is a day-by-day plan, 10 is judge Q&A prep.

Team split (4 people) — each owns a vertical slice end-to-end rather than a single layer, since with 4 people you can't afford pure hardware/software silos:

- **Person A — Crowd side:** crowd band firmware, BLE gateway firmware, NFC bind/reset flow, exit-routing (pathfinding) logic.
- **Person B — Remote side:** remote band firmware (GPS + fall detection), LoRa gateway, route-deviation logic, trekker companion app.
- **Person C — Backend + AI:** Flask/DB/API, geofencing + anomaly fusion, CCI/YOLO integration (reused from CrowdPulse — this is wiring, not from-scratch modeling).
- **Person D — Dashboard + demo:** authority dashboard (live map, alerts, E-FIR), registration kiosk UI, crowd-band companion app (if built), and owns pulling the demo script/pitch together.

This split means each person can build and test their slice mostly independently in week 1, then the group converges for integration (section 7) — with only 4 people, protect that integration time; don't let it get squeezed by any one slice running over.

---

1. Goals & Non-Goals

Goals (must demo live)

1. A **crowd band** that vibrates + lights a directional LED when its zone is flagged dangerous, with a working SOS button.
2. A **remote band** that gets a GPS fix, detects a simulated fall, and sends an SOS over LoRa with no cell/WiFi involved.
3. A **backend** that ingests both band types into one tourist/zone/incident data model.
4. **Geofencing + anomaly fusion** (zone breach, fall, no-movement, route deviation) triggering real alerts.
5. The **CCI engine** (reused from CrowdPulse) feeding live/simulated camera input to produce a risk score that drives crowd-band alerts.
6. A **dashboard** showing a live map/zone view, alert feed, and one-click E-FIR generation.
7. NFC-based band identity write/reset flow (tap-to-bind, tap-to-release) at a mock "registration counter."

Non-goals (explicitly do NOT build — say this to judges proactively)

- Real DigiLocker/Aadhaar API integration — **stub it**.
- Real drone hardware — **simulate** dispatch with a mocked "drone en route" state on the dashboard.
- Production-grade encryption/anti-clone NFC — note as future scope.
- nRF52 production PCB — ESP32 dev boards are the demo hardware.
- Real trained ML model with large datasets — RandomForest on synthetic/rule-derived labels is fine; be honest about this.

---

2. User Roles

Role	Interface	Core need
Crowd visitor	Physical band only (no app required)	Feel a warning, see which way to go, press SOS if needed
Trekker	Remote band + companion app	Confirm SOS sent, share location with family, see checkpoint list
Venue authority / control room staff	Web dashboard	See live map, see alerts ranked by severity, dispatch response, file E-FIR
Registration staff	Mock kiosk (can be a laptop page)	DigiLocker-stub login, collect fee, write NFC band, refund/reset on exit

---

3. Hardware — Bill of Materials & Wiring

3.1 Crowd Band (prototype unit)

Component	Purpose	Notes
ESP32 dev board (e.g. ESP32-WROOM-32)	MCU + BLE	nRF52 is the "production" story, ESP32 is your demo reality — say this explicitly
Vibration motor (coin/pancake type) + NPN transistor driver (e.g. 2N2222) + flyback diode	Haptic alert	Motor draws more current than a GPIO can source directly — always driven via transistor
2–3 LEDs (different colors or positions = different exit directions) + resistors (220Ω)	Directional cue	Simplest version: LED-Left / LED-Right / LED-Straight
Tactile push button + 10kΩ pull-down resistor	SOS	Debounce in firmware (see 4.1)

CR2032 coin cell + holder, or small LiPo + regulator for demo convenience	Power	CR2032 is the "target" story; a small LiPo is more forgiving for a live demo (won't brown out mid-pitch)
NTAG213/215 NFC sticker tag	Identity binding	Passive, no power draw, read/written externally
Wristband enclosure (3D printed or off-the-shelf silicone band + project box)	Housing	Doesn't need to be pretty, needs to survive being handled on stage

Wiring summary: Motor → transistor base (via resistor) → ESP32 GPIO; LEDs → GPIO + resistor → GND; Button → GPIO + pull-down → 3.3V; NFC tag is passive (no wiring, just placed near a separate reader at the "gateway/kiosk," not read by the band's own MCU unless you add a PN532 to the band itself — **don't**, that adds cost/complexity to the "dumb band"; the tag is written once at registration and simply carries the UUID that BLE broadcasts reference).

3.2 BLE Gateways (2–3 units for demo)

Component	Purpose
ESP32 dev board ×2–3	Acts as a BLE scanner/receiver, one per "zone"
USB power (power bank or wall adapter)	Gateways sit stationary during demo
(Optional) PN532 NFC reader/writer module	For the registration-counter unit specifically — only ONE of your ESP32s needs this, to demo the tap-to-bind/reset flow

3.3 Remote Band

Component	Purpose
Heltec WiFi-LoRa-32 (V3)	MCU + LoRa radio combined — saves wiring vs separate MCU+LoRa module
NEO-6M GPS module	Location fix, UART interface
MPU6050 accelerometer/gyro	Fall detection, I2C interface
Push button	Manual SOS
Buzzer (piezo)	Local audible alert
LiPo battery + TP4056 charge module	Power
Second Heltec board	Acts as the LoRa **gateway/receiver**, connects to laptop backend over USB serial or WiFi

Wiring summary: GPS TX → Heltec RX2 (UART), GPS RX → Heltec TX2; MPU6050 SDA/SCL → Heltec I2C pins; Button → GPIO + pull-down; Buzzer → GPIO (+transistor if it draws too much current for direct drive — check your buzzer's datasheet).

3.4 Crowd AI (CCI) Rig

Component	Purpose
USB webcam or Pi Camera	Video feed
Raspberry Pi 4 (4GB+) or Jetson Nano	Edge inference (YOLOv8 + CCI + RandomForest)
(If reusing CrowdPulse) existing trained weights/scripts	Don't retrain from scratch — reuse

If Pi/Jetson setup is too much to bring on-stage reliably, **run this on a laptop instead for the live demo** and frame the Pi/Jetson as the "production edge deployment target" — this is a completely legitimate thing to say to judges and de-risks your demo a lot.

---

4. Firmware / Embedded Software

4.1 Crowd Band Firmware (ESP32, Arduino/C++)

Core loop responsibilities: 1. Broadcast BLE advertisement packets containing the band's bound UUID (written at registration) at a fixed interval (~1Hz is enough for zone-level presence). 2. Listen for a short BLE characteristic write/notify from the nearest gateway telling it: `{zone_status: safe|warning|danger, exit_direction: L|R|S}`. 3. On receiving `warning` / `danger`: drive vibration motor (pulse pattern differs by severity — e.g. single buzz for warning, continuous for danger) and light the LED matching `exit_direction`. 4. On SOS button press (debounced, ~50ms): immediately send a high-priority BLE notification/write to the nearest gateway with `{uuid, type: SOS, timestamp}`, and give physical feedback (all LEDs flash) so the wearer knows it registered. 5. Low-power consideration: since this needs to run all day at a venue, deep-sleep between BLE broadcast intervals if your BLE stack allows it — not critical for the demo, but worth one line in the pitch as a "battery life" answer.

Bind/reset behavior: the band itself doesn't need bind/reset logic — the UUID is fixed at manufacture or set via a one-time provisioning step (e.g. a debug serial command during setup), and what actually changes per-visitor is the **backend mapping** of UUID → tourist identity, not the band's firmware. This keeps the band genuinely dumb. (See section 6.4 for how the NFC/reset flow works on the backend side.)

4.2 BLE Gateway Firmware (ESP32)

1. Continuously scan for BLE advertisements from bands in range; maintain a rolling list of `{uuid, rssi, last_seen}`.
2. Forward this list to the backend every 1–2 seconds via WiFi (HTTP POST or MQTT publish) — this is how the backend knows "which UUIDs are in which zone right now."
3. Receive zone-status updates pushed from the backend (via MQTT subscribe or polling) and relay `{zone_status, exit_direction}` to bands via BLE characteristic writes/notifications.

4. Handle SOS relay from bands → immediate uplink to backend, don't wait for the normal polling cycle.

4.3 Remote Band Firmware (Heltec ESP32-LoRa)

1. Poll GPS module (NEO-6M) at a low frequency (e.g. every 30–60s) to conserve power — parse NMEA sentences for lat/long.
2. Continuously read MPU6050; run a simple **fall-detection heuristic**: free-fall detected as a sharp acceleration-magnitude drop toward ~0g followed by a sharp spike (impact), then check for **no significant movement for N seconds after** — this last check is what separates "fell down" from "jumped/stumbled but is fine," and it's worth explicitly describing this two-stage check to judges since a naive single-threshold accelerometer trigger produces constant false positives.
3. On fall-detected OR SOS button press: package `{band_id, lat, long, event_type, timestamp}` and transmit over LoRa to the paired gateway.
4. Buzzer/local feedback on event trigger and on send-confirmation (if the gateway ACKs).

4.4 LoRa Gateway (second Heltec board)

1. Listen continuously for LoRa packets from remote bands.
2. On receipt, forward to backend over USB-serial (simplest for a demo — laptop reads serial) or WiFi if the gateway has connectivity at that point.
3. This is literally your "checkpoint post" concept scaled down to one unit for the demo — say explicitly in the pitch that production deployment means multiple such posts spaced along a trail, meshed back to a point with real backhaul (see PRD section on future scope / your own doc's honesty notes).

---

5. Backend

5.1 Data model (minimum viable)

- **Tourist**: `id (UUID)`, `band_type (crowd|remote)`, `band_id`, `name/identity_ref (stub, not real KYC data)`, `registered_at`, `status (active|exited)`
- **Zone**: `id`, `name`, `type (safe|exit|danger-capable)`, `geo/graph_position`, `current_status`
- **Gateway**: `id`, `zone_id`, `type (BLE|LoRa)`, `last_heartbeat`
- **Telemetry** (high-volume, can be lightweight/ephemeral): `tourist_id`, `zone_id_or_gps`, `timestamp`, `rssi (if BLE)`
- **Incident**: `id`, `tourist_id`, `type (SOS|fall|zone-breach|no-movement|route-deviation|crowd-risk)`, `severity`, `location`, `timestamp`, `status (open|acknowledged|resolved)`, `efir_generated (bool)`
- **CCI reading**: `zone_id`, `timestamp`, `risk_score`, `person_count_estimate`

5.2 API surface (Flask REST + MQTT)

- `POST /gateway/heartbeat` — gateway telemetry ingest (BLE zone-presence lists, or LoRa event packets)
- `POST /incident` — created either by gateway relay (SOS/fall) or by the anomaly/CCI engine internally
- `GET /dashboard/live` — current tourist positions (zone-level or GPS), current zone statuses, open incidents
- `POST /registration/bind` — links a paid/verified tourist to a band UUID (writes NFC + creates Tourist record)
- `POST /registration/release` — exit flow: marks tourist `exited`, invalidates UUID mapping, triggers NFC wipe/reset instruction to the kiosk device
- `POST /incident/{id}/efir` — auto-generates E-FIR text from incident + tourist + location data, one click from dashboard

5.3 Geofencing + anomaly fusion logic

Rule-based fusion engine, runs on every telemetry update:

- **Zone breach**: tourist's current zone not in their permitted/expected zone set → raise `zone-breach` incident.
- **No-movement**: same zone/GPS position (within tolerance) for > threshold time with no BLE/GPS update change → raise `no-movement`, severity scaled by duration.
- **Route deviation** (remote band only): compare current GPS trace against the expected trek path (a predefined polyline) — distance-from-path over a threshold → raise `route-deviation`.
- **Fall**: relayed directly from remote band firmware, trusted as-is (already debounced on-device) → raise `fall`, high severity.
- **Crowd risk**: not per-tourist, it's per-zone, fed by the CCI engine's `risk_score` crossing a threshold → push `zone_status: warning|danger` to all gateways in that zone, which relay to bands (this is the loop back into section 4.2 step 3).

5.4 Pathfinding (your BLE-checkpoint routing idea)

- Model venue as a **graph**: nodes = zones, edges = walkable adjacency, defined once at setup.
- On a `danger` zone status, reweight/remove that node's edges and run **Dijkstra/A*** from each affected zone to the nearest zone marked `type: exit`.
- Result feeds directly into `exit_direction` sent to gateways in section 4.2 step 3 — this is the mechanism behind your "exit load-balancing" claim.
- Implementation: this is a small, well-known algorithm — use `networkx` in Python on the backend rather than hand-rolling Dijkstra; keep the graph as a static JSON config editable by venue admins.

---

6. ML / AI Systems

6.1 CCI (Crowd Compression Index) — reused, not rebuilt

- Input: camera frame → YOLOv8 person detection → bounding boxes.
- CCI computation (your patented module — reuse as-is from CrowdPulse) turns detections into a density/compression metric per zone over time.
- Output: `risk_score` per zone per time window, fed into RandomForest (already trained in CrowdPulse) to classify `safe|warning|danger`.
- **Your job here is integration, not re-invention**: wire CCI's output into the new `/incident` and zone-status pipeline described in 5.3,

don't rebuild the model.

6.2 Fall detection — rule-based (not ML) for the demo

- As described in 4.3 — free-fall + impact + subsequent stillness. This is intentionally rule-based, not ML, because it's explainable, works with zero training data, and is honestly good enough at this fidelity. If asked "why not ML," the honest answer is: rule-based thresholds are actually standard practice for wearable fall-detection at this scale, and an ML model needs labeled fall data you don't have time to collect — say this plainly, it's a stronger answer than pretending you trained a model you didn't.

6.3 Route deviation — geometric, not ML

- Distance from a predefined GPS polyline. No model needed. Simple and defensible.

6.4 NFC bind/reset flow (ties identity to a physical band)

1. **Registration:** tourist completes DigiLocker-stub login + pays fee at kiosk UI → backend creates `Tourist` record with a new UUID → kiosk's PN532 module writes that UUID onto the band's NTAG tag → band is now handed to the tourist.
2. **In use:** the UUID on the tag is *not* actively used by the crowd band during operation (the band already knows its own UUID in firmware/config) — the NFC write step is really about **staff-facing verification** (tap a band at any point to see whose it is / re-confirm binding), and about resetting.
3. **Exit/reset:** tourist returns band at exit → kiosk taps it → backend calls `/registration/release` → tourist marked `exited` → **kiosk writes a blank/new placeholder UUID onto the tag**, or simply overwrites it with a fresh UUID for the *next* visitor, and the backend pre-registers that fresh UUID as "unbound, available."
4. **Privacy note to state explicitly to judges:** the tag itself never carries name/Aadhaar/phone — only a random UUID. All real identity lives server-side, linked temporarily. This is a genuinely good privacy design and worth stating as a deliberate choice, not an accident.

---

7. Integration Plan (how the pieces actually connect)

Build order matters — integrate bottom-up so nothing is developed in isolation until demo day:

1. **Week 1:** Each hardware piece works standalone (crowd band vibrates on command via serial, remote band gets a GPS fix, gateway prints scanned BLE UUIDs to serial monitor).
2. **Week 1–2:** Backend skeleton up — DB schema, REST endpoints stubbed with fake data, dashboard shows fake data (this lets frontend/backend/dashboard people work in parallel with hardware people).
3. **Week 2:** Gateway → Backend real connection (BLE presence lists actually posting to `/gateway/heartbeat`); LoRa gateway → Backend serial bridge working.
4. **Week 2–3:** Anomaly fusion logic running against real telemetry; CCI integration wired into zone-status pipeline.
5. **Week 3:** Full loop test — trigger a fall on the remote band, confirm it appears as an incident on the dashboard within a few seconds; trigger high CCI on camera feed, confirm crowd bands actually buzz.
6. **Final days:** NFC bind/reset flow, pathfinding/exit-routing, E-FIR generation, dashboard polish, rehearse the demo script (section 8) end-to-end at least 5 times.

Biggest integration risk: live BLE + live camera + live LoRa all running simultaneously on stage WiFi is fragile. Mitigate by having a **recorded-video fallback** for the CCI/camera part and a **pre-scripted incident trigger button** on the dashboard that simulates any sensor path if live hardware flakes — judges care about seeing the *system* work, and a graceful "simulate" fallback is normal and expected at hackathons, not a red flag, **as long as you're upfront that it's a fallback**.

---

8. Demo Script (3 minutes, matches your doc's section 8)

Time	Beat	What's physically happening
0:00–0:20	Hook	One line: two deadly failure modes, one system, predictive not reactive
0:20–0:40	Registration	Live: tap DigiLocker-stub login, band gets NFC-bound on stage
0:40–1:20	Scene 1 (Remote)	Simulate a fall on the remote band on-stage → buzzer fires → dashboard flashes incident + GPS pin within seconds → mention drone dispatch as simulated/future
1:20–2:10	Scene 2 (Crowd)	Camera feed (live or pre-recorded) shows rising density → CCI risk score climbs on dashboard → crowd band on stage buzzes + LED points to exit → mention exit load-balancing via the zone graph
2:10–2:40	Dashboard + E-FIR	One click to generate E-FIR from the incident just created — show the actual generated text
2:40–3:00	Close	Restate the novelty triad: patented CCI + dual-environment + genuinely affordable band, tie back to SDG 10/11

---

9. Suggested Day-by-Day Plan (assuming ~10–14 day build window — adjust to your actual timeline)

Days	Focus
1–2	Hardware bring-up: each board powers on, each sensor gives readable output on serial monitor, no networking yet
3–4	Backend schema + stub API + dashboard skeleton (fake data) in parallel with firmware BLE/LoRa basics

5–6	Gateway ↔ Backend real data flow (BLE presence, LoRa relay)
7–8	Anomaly fusion + geofencing logic; CCI integration from CrowdPulse
9	Pathfinding/exit-routing + NFC bind/reset flow
10	E-FIR generation + dashboard polish
11–12	End-to-end integration testing, fix the inevitable breakage
13	Full dry-run rehearsals (at least 3), build the fallback/simulate paths
14	Buffer day + final rehearsal + pack hardware

---

10. Judge Q&A Prep (direct, honest answers — your own doc already models this well in section 10, extending it here)

- **"Is this actually predictive or just faster reactive?"** — CCI predicts *rising* compression before a threshold incident, and route-deviation/no-movement catch developing problems before they become emergencies; be ready to explain the CCI risk-score-over-time curve, not just a single threshold alarm.
- **"What happens with zero network at all, even LoRa?"** — Be honest: LoRa needs a gateway/post within range; total off-grid with zero infrastructure isn't solved today, satellite uplink is named as future scope.
- **"How is the ₹30–40 band real?"** — At-scale BOM target, hackathon unit costs more (ESP32-based), be upfront with the ~₹50–100 realistic number, as your own doc already says.
- **"Isn't the disposable band e-waste?"** — Refundable-deposit reusable band + a genuinely disposable ₹5 printed-QR ID-only tier for those who don't want the deposit.
- **"Is the NFC secure?"** — Tag carries only a random UUID, not identity data; anti-clone crypto tags are future scope, be upfront that a bare UID is technically clonable today.
- **"Why no ML for fall detection?"** — Rule-based two-stage check (free-fall + impact + stillness) is explainable, needs no training data, and is standard practice at this scale; ML is future scope once real fall data exists.
- **"Isn't this just CrowdPulse re-skinned?"** — CCI is one reused module, not the whole system; the band architecture, geofencing, dual-environment split, and NFC identity flow are new and purpose-built for this problem statement.

---

11. Open Decisions For Your Team (decide before you build, not during)

1. **Kiosk-based DigiLocker login (staff-assisted) vs. pre-registration on tourist's own phone before arrival** — changes whether you build kiosk UI or mobile pre-reg flow. Pick one for the demo.
2. **CCI camera feed: live webcam on stage vs. pre-recorded crowd footage** — live is more impressive if reliable, recorded is safer. Decide based on how confident you are in stage WiFi/lighting.
3. **How many BLE gateways to actually bring** — 2 is enough to show zone-to-zone difference; 3 lets you show routing around a "closed" middle zone, which is a stronger demo of the pathfinding feature specifically.